

Consistent error responses



Wrap every API error in the same shape, so clients always know where to look.

The big idea

Things go wrong in every API: a missing field, a task that doesn't exist, a user who isn't logged in. The server still answers, with a **status code** like **404** and a JSON body.

DRF's default error bodies come in **different shapes**. Your **exception handler** is a help desk that files every complaint on the **same form**.

What your code receives

The tests pass an exception in as `exc`, and an empty dict `{}` as `context`.

```
from rest_framework import exceptions

# test 2 does this:
response = api_exception_handler(exceptions.NotFound(), {})

# other exceptions the tests pass in as exc:
exceptions.ValidationError({"title": ["This field is required."]})
exceptions.NotAuthenticated() # 401
exceptions.MethodNotAllowed("POST") # 405
KeyError("boom") # a bug, not a web error
```

Two shapes in, one shape out

Same status code, but every body gets the "error" envelope.

```
# DRF's default bodies (before your change)
{"title": ["This field is required."]}      # ValidationError, 400
{"detail": "Not found."}                   # NotFound, 404

# What your handler must return instead
{"error": {
    "status": 400,
    "message": "Invalid input.",
    "details": {"title": ["This field is required."]}},
}}
{"error": {"status": 404, "message": "Not found.", "details": {}}}
```

New words

exception handler

A function that turns an exception into a response. DRF calls it.

status code

400 bad input, 401 not logged in, 403 not allowed, 404 not found.

500

A server bug. Unexpected errors should stay 500s.

ValidationError

"The data you sent is wrong", with messages per field.

envelope

A fixed outer shape around every body, like `{"error": {...}}`.

Response

DRF's reply object, with `.status_code` and `.data`.

Tool: DRF's exception_handler

Already called on the first line of your function. A bug gives back `None`.

```
from rest_framework import exceptions
from rest_framework.views import exception_handler
response = exception_handler(exceptions.NotFound(), {})
print(response.status_code)
print(response.data)
print(exception_handler(KeyError("oops"), {}))    # a bug, not a web error
```

OUTPUT

404

`{'detail': ErrorDetail(string='Not found.', code='not_found')}`

`None`

Tools: str and isinstance

```
from rest_framework import exceptions
from rest_framework.views import exception_handler

response = exception_handler(exceptions.ParseError(), {})
print(str(response.data["detail"]))      # plain text

# isinstance asks: "is this value that kind of thing?"
bad = exceptions.ValidationError("bad")
print(isinstance(bad, exceptions.ValidationError))
print(isinstance(exceptions.NotFound(), exceptions.ValidationError))
```

OUTPUT

```
Malformed request.
True
False
```

Dots vs square brackets

`response.status_code`

- An **attribute** of the response object
- Read it with a **dot**
- A number, like `404`
- Use it for `"status"`, never type `400`

`response.data["detail"]`

- A **key** inside a dictionary
- Read it with **square brackets**
- An `ErrorDetail`: wrap it in `str(...)`
- Missing for validation errors!

Pattern: choose, then wrap

The same shape your handler needs, on different data:

```
order = {"item": "tea", "problem": "out of stock"}

if order["problem"] == "out of stock":
    note, extra = "Sorry, we ran out.", {"item": order["item"]}
else:
    note, extra = order["problem"], {}

reply = {"reply": {"code": 7, "note": note, "extra": extra}}
print(reply)
```

OUTPUT

```
{'reply': {'code': 7, 'note': 'Sorry, we ran out.', 'extra': {'item': 'tea'}}}
```

Your task, step by step

- 1 Keep the first line; if `response` is `None`, return `None` → **test 6**
- 2 `ValidationError`: message `"Invalid input."`, details `response.data` → **test 1**
- 3 Otherwise: message `str(response.data["detail"])`, details `{}` → **tests 2-4**
- 4 Build `{"status": ..., "message": ..., "details": ...}`
- 5 Set `response.data = {"error": ...}`; status is `response.status_code` → **tests 1-5**
- 6 Return the same `response` object → **tests 1-6**

Common mistakes

- Reading `["detail"]` on a validation error: `KeyError`. Check it first
- Returning your dict instead of `response`: the tests need `.status_code`
- Typing `"status": 400` by hand: test 5 checks 405, 429 and 400
- Forgetting the outer `"error"` key
- Turning `None` into a response: real bugs should stay 500s

Where you'll use this

A React app shows a red banner whenever an API call fails. With one error shape, it only needs `body["error"]["message"]`, whatever went wrong.

- Switch it on once, with the `EXCEPTION_HANDLER` setting
- Mobile apps and other services get the same predictable errors
- Unexpected bugs still become 500s, which you log and fix

Check yourself

1. What does `exception_handler` give back for a `KeyError`, and why?
2. Why must you check for `ValidationError` before reading `"detail"`?
3. Why use `response.status_code` instead of typing the number?
4. Why return `response`, not the dictionary you built?

Keep learning

Errors and Exceptions

Python docs · tutorial · beginner. Start here: a refresher on raising and catching exceptions, since an exception handler is just code that receives one.

HTTP response status codes

MDN Web Docs · docs · beginner. The meaning of 400, 401, 403, 404 and 500, so every error your handler shapes keeps the right status.

Exceptions: custom exception handling

Django REST framework · docs · intermediate. Official guide to writing a handler that calls DRF's `exception_handler` and reshapes its response, as this exercise does.

RFC 9457: Problem Details for HTTP APIs

IETF RFC Editor · docs · intermediate. The web standard for a consistent JSON error shape; skim it later to see how real APIs agree on error formats.